

AN OPTIMAL FEATURE SELECTION AND RECOGNITION FOR NETWORK INTRUSION DETECTION SYSTEM USING MACHINE LEARNING MODELS

Submitted

By

PRASENJIT JANA

BACHELOR OF TECHNOLOGY

In

COMPUTER SCIENCE AND ENGINEERING

ABSTRACT

Network intrusion detection systems (NIDS) play a vital role in protecting networks from an ever-evolving landscape of cyber threats”. These systems aim to identify malicious activities and prevent potential damage to critical infrastructure. Despite their importance, existing NIDS face significant challenges, including high false alarm rates, difficulty in handling high-dimensional data, and limited ability to detect novel or zero-day attacks. These limitations reduce their reliability and overall effectiveness in real-world scenarios. To overcome these challenges, our project introduces a novel method to detect malicious activity in the network. The proposed novel approach uses SMOTE to handle imbalance in the dataset followed by feature extraction using Pearson correlation coefficient to obtain the eminent features. Feature Selection is performed by using filter method MRMR (Maximum Relevance Minimum Redundancy) to reduce the dimensionality and to remove the irrelevant features from the network intrusion dataset. Later we will evaluate the performance using different machine learning techniques like Support Vector Machines (SVM), K-Nearest Neighbor(KNN), (Decision Tree(DT)). We will choose the best evaluated machine learning model that detects the intrusions and that reduces the “false alarm rate”. The results of the proposed method will be compared with state of art techniques by using the performance metrics like accuracy, precision, recall, and F1-score, making it a promising solution for detecting sophisticated network intrusions.

Keywords:

Network Intrusion Detection System, SMOTE, Pearson Correlation, MRMR, Machine Learning, SVM, KNN, Decision Tree

INDEX	Page No
1. INTRODUCTION	01
1.1 Introduction of Intrusion Detection System	01
1.2 Existing System and Disadvantages	02
1.3 Proposed System and Advantages	05
1.4 Objectives of the Project	07
1.5 Organization of Project	08
2. REQUIREMENTS ANALYSIS	11
2.1 Introduction	11
2.2 Hardware & Software Requirements	11
2.3 Software & Hardware Requirements Specification	12
3. LITERATURE SURVEY	14
3.1 Related Works	14
3.2 Survey Table	15
4. MODULES	17
4.1 Introduction	17
4.2 Module Implementation	18
5. SYSTEM DESIGN	21
5.1 Introduction	21
5.2 Process Flow of Intrusion Detection System	25
5.3 Flowchart Diagram	27
5.4 UML Diagrams	30
6. SYSTEM IMPLEMENTATION	34
6.1 Introduction	34
6.2 Selection Software	35
6.3 Steps Involved in Implementation of Algorithm/Technology	
6.4 Sample Code	38
7. TESTING	40
7.1 Introduction	40
7.2 Test cases	42

8. SCREENS & REPORTS	44
8.1 Introduction	44
8.2 Screenshots	44
9. CONCLUSION and FUTURE SCOPE	48
9.1 Conclusion	48
9.2 Future Scope	48
10. BIBILOGRAPHY	49
10.1 References	49
10.2 Web Links	51

LIST OF FIGURES

Figure No.	Name of the Figure	Page No.
5.1	System Architecture of NIDS	23
5.2	Flow chart for system design	27
5.3	Usecase diagram for NIDS	31
5.4	Sequence Diagram for NIDS	33
8.1	SVM Metrics	44
8.2	Decision Tree Metrics	45
8.3	KNN Metrics	46
8.4	Comparison	47

LIST OF TABLES

Table No.	Name of the Table	Page No.
3.1	Research Work	15
7.1	Data Preprocessing Validation	42
7.2	Feature Selection Accuracy (MRMR)	42
7.3	Machine Learning Model Performance	43

1. INTRODUCTION

In today's digital era, cyber security has become a critical concern for organizations, governments, and individuals. The increasing reliance on computer networks for data storage, communication, and financial transactions has made them vulnerable to various cyber threats. Network Intrusion Detection Systems (NIDS) play a crucial role in identifying and mitigating these threats to ensure the security and integrity of network infrastructure. Among the most effective tools for safeguarding network infrastructure are Network Intrusion Detection Systems (NIDS). NIDS play a pivotal role in identifying, analyzing, and mitigating malicious activities by continuously monitoring network traffic for suspicious patterns. These systems employ various techniques such as signature-based detection, anomaly detection, and machine learning algorithms to detect potential security breaches in real-time. By leveraging NIDS, security administrators can gain valuable insights into network behavior, allowing them to promptly respond to potential intrusions. Modern NIDS solutions are equipped with advanced features like automated alerts, threat visualization dashboards, and integration with security information and event management (SIEM) systems to enhance threat detection capabilities. The development of an optimal NIDS requires careful selection of features and appropriate machine learning models to effectively distinguish between legitimate network traffic and malicious activities. This ensures not only high detection accuracy but also a reduction in false alarm rates, which is critical for efficient security management.

1.1 Introduction of Intrusion Detection System

A Network Intrusion Detection System is a security mechanism designed to monitor network traffic for suspicious activities and potential security breaches. NIDS can detect different types of attacks, such as denial-of-service (DoS), unauthorized access, malware propagation, and other malicious activities. Traditional rule-based intrusion detection approaches have proven to be less effective against modern and sophisticated cyber-attacks due to their static nature and reliance on predefined signatures. This has led to the adoption of machine learning (ML) models for intrusion detection, offering enhanced adaptability and accuracy. Feature selection plays a vital role in improving the efficiency and performance of machine learning-based intrusion detection systems. Feature selection involves identifying the most relevant features from large datasets to optimize the detection process while reducing computational complexity. An optimal feature selection technique ensures that only the most significant attributes are utilized, thereby enhancing model accuracy and minimizing processing time.

Machine learning techniques, including supervised, unsupervised, and deep learning models, have been increasingly employed to develop intelligent and automated intrusion detection systems. Supervised learning methods such as decision trees, random forests, support vector machines (SVM), and deep neural networks have demonstrated high accuracy in intrusion detection by learning from labeled datasets. Unsupervised learning approaches, including clustering techniques like K-means and auto encoders, help in detecting anomalies in network traffic without prior knowledge of attack signatures. The objective of this project is to design and implement an optimal feature selection and recognition mechanism for network intrusion detection using machine learning models. By leveraging advanced feature selection algorithms and robust ML classifiers, the system aims to enhance detection accuracy while reducing false positives and computational overhead. The integration of feature selection techniques, such as Principal Component Analysis (PCA), Recursive Feature Elimination (RFE), and mutual information-based methods, will contribute to an efficient intrusion detection framework.

Furthermore, the project will explore real-world datasets such as the KDD Cup 99, NSL-KDD, or CICIDS 2017 for training and testing the intrusion detection model. The effectiveness of the proposed system will be evaluated based on performance metrics such as accuracy, precision, recall, and F1-score.

1.2 Existing System and Disadvantages

Conventional network intrusion detection systems (NIDS) are largely based on signature-based, anomaly-based, and rule-based approaches. These techniques have been largely employed for the detection of intrusion but are faced with numerous weaknesses that limit their efficiency in today's cyber-security environment.

The signature-based detection mechanism compares network traffic against a set of known attack signatures. It is highly accurate in detecting previously seen attacks but does not perform well against zero-day attacks and new patterns of attacks without pre-existing signatures. The method also needs frequent updates to the signature database in order to continue being effective. Polymorphic and metamorphic malware are commonly used by attackers to alter malicious code structures so that they can evade signature-based systems. This method also relies greatly on existing knowledge of attacks and makes systems susceptible to new and slightly modified attacks.

The anomaly-based detection technique creates a profile of normal network activity and marks anomalies as possible threats. Although this method is helpful in identifying unknown attacks, it is vulnerable to high false positives because of the dynamic nature of network traffic. Ordinary user behavior may change drastically with software updates, changes in workload, or fluctuations in seasonal traffic, causing misclassifications. The difficulty in establishing what is "normal" network behavior makes it difficult to detect anomalies, and there is a need for ongoing tuning and retraining to ensure accuracy. The computational overhead of continuously monitoring traffic patterns may affect system performance, rendering real-time detection challenging. Attackers can also design network traffic that looks similar to normal behavior in order to avoid detection, further reducing the effectiveness of this method.

Rule-based systems are used by many conventional NIDS, where there are pre-existing rules that specify whether network traffic is malicious or normal. While rule-based detection is effective at detecting known patterns of attacks, it needs manual updates and upkeep to remain useful against changing threats. The static character of rule sets renders them unsuitable for identifying subtle attacks that exploit new vulnerabilities. Further, evasions like encryption, traffic fragmentation, and protocol manipulation can be employed by attackers to evade these pre-configured rules. With expanding networks in size and complexity, the rule base also expands, causing scalability problems and performance bottlenecks. The use of human-defined rules also renders this method less responsive to new cyber threats.

To address the shortcomings of conventional methods, machine learning-based systems have been incorporated into certain contemporary NIDS. These systems examine network traffic and learn patterns to differentiate between normal and malicious behavior. Although machine learning methods provide enhanced adaptability and detection, they are not without their own challenges. The computational cost of training and executing machine learning models usually prevents real-time intrusion detection, as there needs to be a high level of processing power in order to inspect traffic effectively.

In general, current NIDS solutions are severely hampered in accommodating contemporary cyber-attacks. The advanced sophistication of attack methods, dynamic nature of traffic in networks, and necessity of real-time behavior call for sophisticated detection mechanisms to address the limitations of conventional and machine learning-based solutions. The perfect intrusion detection system would aim to strike a balance between detection accuracy,

efficiency, and adaptability while being as low on false positives and computational overhead as possible..

Disadvantages:

Although ubiquitous, conventional and early machine learning-driven intrusion detection systems are plagued by numerous deficiencies that undermine their capability to function effectively in contemporary cyber defense environments.

1. One of the biggest problems is the high rate of false alarms that are common in most of today's intrusion detection systems. Anomaly detection techniques, while capable of detecting new threats, are prone to labeling normal network traffic anomalies as malicious. This results in an excessive amount of false alarms, and the security team is subjected to alert fatigue as it spends precious time scanning non-threat events. Failure to truly distinguish non-threatening departures from true cyber-attacks reduces the overall system's reliability and effectiveness.
2. But another essential limitation is the ineffective detection of zero-day attacks. Signature-based intrusion detection systems rely on known attack patterns and therefore cannot identify new and unknown cyber-attacks. Malware can be easily altered by attackers or new unknown vulnerabilities are exploited to avoid detection. The use of historical data renders signature-based solutions ineffective in identifying new attack vectors, which provides an opportunity for a security vulnerability that can be exploited by advanced attackers.
3. Scalability problems also inhibit the performance of conventional NIDS, especially with traffic loads on the network continuing to expand exponentially. Conventional intrusion detection is usually not able to process huge amounts of data efficiently, so threats get detected with a delay. Real-time intrusion detection is increasingly hard with larger networks, as it takes more computing power to analyze high-speed streams of data. Without optimized processing algorithms, such systems become impractical for high-speed and large networks, making them less effective overall.
4. The second crucial challenge is managing high-dimensional data. Network traffic data consists of different features like packet features, flow features, and protocol features. Intrusion detection models are not normally capable of managing such intricate, high-dimensional data. Ineffective feature selection and data preprocessing techniques add computational complexity, detection time, and model accuracy. Insufficient effective dimensionality reduction techniques might hinder intrusion detection systems from extracting useful patterns from large amounts of data, and consequently, performance further.

1.3 Proposed System and Advantages

To overcome the shortcomings of existing intrusion detection systems, we propose an optimized feature recognition and selection system that integrates state-of-the-art machine learning algorithms with efficient data preprocessing techniques. The proposed system aims to improve detection rates, reduce computational overhead, and speed up real-time threat detection. By incorporating optimized feature selection, data balancing techniques, hybrid machine learning algorithms, and deep learning-based anomaly detection, the proposed framework provides an improved scalable and efficient network security solution.

One of the most important aspects of this system is feature selection through Minimum Redundancy Maximum Relevance (MRMR). Traditional NIDS models process large, high-dimensional data, which adds greater computational complexity and longer response times. MRMR solves this issue by keeping only the most informative and relevant features and removing redundant or irrelevant attributes. Focusing on the most relevant features of network traffic, MRMR enhances the accuracy of classification and enables faster data processing, making the system more efficient for real-time intrusion detection.

To further pre-process data, Synthetic Minority Over-sampling Technique (SMOTE) is used to balance the dataset. The majority of intrusion detection datasets suffer from extreme class imbalance where some attack types occur much less frequently than others. Such imbalance usually results in biased models with high accuracy for frequent attack types and poor quality at detecting low-frequency but harmful attacks. SMOTE resolves this to some extent by generating synthetic instances for minority attack types so that the machine learning model learns a more even understanding of network threats. This improves the system to detect a wide range of intrusions while minimizing bias.

To further enhance real-time intrusion detection, sophisticated deep learning models like Convolutional Neural Networks (CNN) and Long Short-Term Memory (LSTM) are integrated into the system. LSTM, being a type of recurrent neural network, is particularly suited to process sequential patterns of network traffic and hence particularly suited for time-domain anomaly detection. CNN, on the other hand, is particularly suited for feature detection and pattern extraction, allowing the system to detect subtle attack signatures. With the integration of these deep learning models, the system is dynamically trained to adapt to evolving cyber-attacks and significantly improves its ability to detect zero-day attacks and unknown intrusion attempts.

Advantages of the Proposed System:

The system described here has a number of benefits over conventional NIDS, addressing major vulnerabilities while enhancing security and efficiency overall.

One of the key benefits is increased accuracy of detection. By the use of optimized feature selection techniques and a hybrid classification model, the system reduces false positives and increases overall dependability. The inclusion of deep learning models also increases classification accuracy, enabling the system to detect complex attack patterns that may be undetected by conventional models. This makes the system a more reliable and efficient intrusion detection system.

Another benefit is effective processing of huge network data at enormous scales. Conventional NIDS is bogged down by the tremendous amounts of traffic on contemporary networks, leading to slow detection and late response times. The suggested system improves processing by feature space reduction using MRMR and taking advantage of cloud-based and distributed computing methods. This provides scalability, which enables the system to operate effectively in high-traffic environments without compromise in performance.

The system is also faster to react to emerging and evolving threats. As compared to the traditional signature-based detection, which is dependent on pre-defined attack patterns, the system here learns new network patterns on a regular basis. The integration of adaptive machine learning and deep learning frameworks provides the system with the ability to update its detection parameters on a continuous basis, improving its ability to identify zero-day attacks. The system enhances its effectiveness in handling evolving cyber threats with its responsiveness.

Lastly, the proposed architecture is scalable and deployable. Compatible with current network infrastructures, the system can be deployed on cloud systems, edge computing systems, and IoT networks. Modular system architecture ensures compatibility with current security systems, and hence, the system is deployable on heterogeneous cyber-security infrastructures from enterprise networks to government systems. Existing NIDS solutions are plagued by a set of issues including high false positives, poor detection of zero-day attacks, and scalability. The system therefore utilizes improved feature selection, hybrid machine learning algorithms, and deep learning-based anomaly detection to enhance accuracy, provide maximum efficiency, and deliver improved real-time threat response. Through these technologies, the system provides a scalable, robust, and future-proof solution to network intrusion detection.

1.4 Objectives of the Project

The primary objective of this project is to develop an optimized network intrusion detection system (NIDS) by leveraging machine learning techniques. The system is designed to improve the accuracy of intrusion detection, reduce false alarm rates, and enhance the adaptability of the model to evolving cyber threats. The detailed objectives of the project are as follows:

1. **To Achieve High Detection Accuracy:** The proposed system aims to improve the accuracy of intrusion detection by utilizing advanced machine learning techniques such as Decision Trees, Random Forest, Support Vector Machines (SVM), and Neural Networks. By employing feature selection techniques like Minimum Redundancy Maximum Relevance (MRMR), the model can focus on the most critical features, eliminating irrelevant or redundant data that may lead to misclassification.
2. **To Reduce False Alarm Rates:** One of the major drawbacks of existing NIDS is the high false positive rate, where normal traffic is mistakenly flagged as an attack. The project aims to incorporate efficient filtering and classification techniques to minimize false positives, ensuring that only legitimate threats trigger alerts.
3. **To Apply Filter-Based Feature Selection Methods:** The project will implement filter-based feature selection techniques to reduce dataset dimensionality, making the intrusion detection process faster and more efficient. By using feature selection algorithms such as Principal Component Analysis (PCA) and Recursive Feature Elimination (RFE), the system will focus on essential features while discarding irrelevant ones.
4. **To Improve the Classification Accuracy of NIDS:** The system will be trained on benchmark datasets such as KDD Cup 99, NSL-KDD, and CICIDS 2017 to improve classification accuracy. By using hybrid models that combine multiple machine learning techniques, the system will be able to enhance its prediction capability.
5. **To Develop a Scalable and Efficient Detection Framework:** The project aims to build a scalable and efficient intrusion detection system that can handle large volumes of network traffic in real-time. It will be designed to function effectively in cloud-based and distributed environments, making it suitable for deployment in enterprise-level security systems.
6. **Implement Data Pre-processing Techniques for Enhanced Performance:** Effective data pre-processing is crucial for achieving high performance in intrusion

detection systems. The project will implement techniques such as Synthetic Minority Oversampling Technique (SMOTE) to handle class imbalances and ensure robust learning across all attack categories.

7. **Compare Performance with Existing Systems:** The proposed system will be evaluated against existing NIDS models to benchmark improvements in accuracy, precision, recall, and F1-score. A comparative analysis will be conducted to demonstrate the efficiency of the proposed feature selection and classification methods.

1.5 Organization of the Project

The project is organized into several chapters, where each chapter responds to an imperative issue of the research, implementation, and assessment of the given NIDS proposal. The organized structure helps in keeping the flow of issue identification to solution implementation and performance assessment straightforward.

The Introduction, which is the first chapter, gives an overview of the project area and mentions the importance of network intrusion detection systems in contemporary cybersecurity. The motivation for the research is highlighted, focusing on the shortcomings of current intrusion detection methods and the need for a better approach. The introduction also defines the problem statement, research goals, and the study area in a very clear manner, laying a good foundation for the rest of the chapters.

The second chapter, Literature Survey, introduces a clear synopsis of existing research on intrusion detection approaches. It includes different detection methods, i.e., traditional, signature-based, anomaly-based, and machine learning-based systems. This chapter also provides an overview of the recent developments in intrusion detection from research articles released in 2024 and 2025. Through the analysis of the strengths and limitations of current methods, it determines major research gaps and explains why there is a need for an optimized feature recognition and selection framework.

The third chapter, Problem Statement and Proposed System, clearly articulates the issues of conventional NIDS, i.e., high false positives, restricted zero-day attack detection, and scalability. It presents the proposed feature selection and machine learning-based detection system to address these shortcomings. The chapter describes the essential methodologies,

such as data pre-processing, feature extraction, and model selection, and sheds light on how the system improves intrusion detection capacity.

The fourth chapter, System Design and Methodology, outlines the architecture of the proposed system. It defines the data flow and processing pipeline, including each step from raw data acquisition to final threat classification. The chapter discusses feature selection methods like Minimum Redundancy Maximum Relevance (MRMR), Principal Component Analysis (PCA), and Recursive Feature Elimination (RFE). Furthermore, it explains the integration of machine learning algorithms, such as Decision Trees, Random Forest, Support Vector Machines (SVM), and Neural Networks. The design approach makes sure that the system is scalable and efficient in processing large-scale network traffic data.

The fifth chapter, Implementation, deals with the implementation aspects of the project. It gives information on the software environment, tools, and programming languages utilized for development. The chapter also describes pre-processing steps for datasets, normalization methods, and imbalanced dataset handling with the help of Synthetic Minority Oversampling Technique (SMOTE). Additionally, it deals with hyperparameter tuning, model training protocols, and optimization methods used to enhance system performance. This chapter is a comprehensive technical guide to building and deploying the proposed NIDS.

The sixth chapter, Experimental Results and Evaluation, provides an extensive analysis of the performance of the system. It performs the suggested model on standard benchmark datasets like KDD Cup 99, NSL-KDD, and CICIDS 2017. The performance of the system is measured using metrics like accuracy, precision, recall, and F1-score. Performance results are compared against state-of-the-art intrusion detection models to emphasize improvements in detection accuracy and computational complexity. This chapter provides graphical representations, confusion matrices, and statistical performance measures to present clear visualization of model efficacy.

Conclusion and Future Scope, the concluding chapter, highlights the contributions made by the research in the context of cyber-security, summarizing research outcomes. This chapter addresses practical implementations of the system proposed herein for real-time network security, its applicability to cloud-based, enterprise, and IoT-based security infrastructures. The chapter also defines future improvements, such as incorporating deep learning-based

hybrid models, real-time anomaly detection mechanisms, and adaptive security methods. These will further enhance the system's capacity to detect dynamic cyber threats.

This organized format guarantees a sequential flow of information, leading the reader through the research process from problem formulation to solution implementation and assessment. Through systematic consideration of each element of the project, the study presents an all-inclusive framework for creating an advanced, efficient, and scalable intrusion detection system.

2. REQUIREMENTS ANALYSIS

2.1 Introduction of Requirements Analysis

The Requirements Analysis stage is an integral part of the development of the suggested Network Intrusion Detection System (NIDS). It is a process of determining the necessary hardware and software elements that are required for the successful installation and deployment of the system. Through the definition of functional and non-functional requirements, this stage guarantees that the system satisfies the most important parameters like performance, security, scalability, and support for real-time processing. A properly formatted requirements analysis serves as the backbone of an efficient, resilient, and flexible intrusion detection system.

The system here proposed is based on using machine learning and deep learning models for detecting network intrusions. The system incorporates feature selection methods, anomaly detection procedures, and classification algorithms to enhance accuracy and decrease false alarm rates. The architecture of the system is such that it can handle high computational performance, real-time processing of data, and scalable storage, which enables it to be deployed in contemporary cyber security systems. For these goals to be realized, the system needs to be deployed in an environment that offers high processing power, sophisticated data handling, and easy integration with machine learning libraries.

2.2 Hardware & Software Requirements

To build and deploy the proposed machine learning-based NIDS, specific hardware and software components are needed. These requirements ensure smooth execution, processing speed, and compatibility with different network environments.

Hardware Requirements:

To handle large-scale network traffic data and process complex machine learning algorithms, the system requires high-performance hardware.

- **Processor:** Intel Core i5 or higher (minimum 3.0 GHz) for optimal processing speed.
- **RAM:** Minimum 8GB (16GB recommended) for handling large datasets efficiently.
- **Storage:** At least 500GB HDD or SSD for fast read/write operations.
- **Graphics Card:** Required if deep learning techniques (CNN, LSTM) are implemented.

- **Network Interface Card (NIC):** Essential for capturing real-time network traffic for intrusion detection.

A system with these configurations ensures that the NIDS operates efficiently, minimizing latency and optimizing performance.

Software Requirements:

The software environment plays a significant role in building and deploying the machine learning-based intrusion detection system. The following software tools and frameworks are required:

- **Programming Language:** Python (preferred for machine learning libraries and frameworks).
- **Development Environment:** Jupyter Notebook, Google Colab, or VS Code.
- **Machine Learning Libraries:** TensorFlow, Scikit-learn, Pandas, NumPy, Matplotlib.
- **Database Management System:** MongoDB or MySQL for storing network logs and intrusion records.
- **Operating System:** Windows 10/11 or Linux

2.3 Software & Hardware Requirements Specification

The Software & Hardware Requirements Specification is crucial for ensuring the successful implementation and performance optimization of the Network Intrusion Detection System (NIDS). These specifications define the essential functional and non-functional requirements, ensuring the system is scalable, secure, and efficient.

Functional Requirements: Functional requirements define the core functionalities that the system must fulfill to detect and prevent network intrusions effectively. These include data collection, preprocessing, classification, and alert generation.

1. Collect and Process Network Data: Extracting relevant features from network traffic using feature selection techniques like MRMR (Minimum Redundancy Maximum Relevance) to remove redundant or irrelevant data. Applying data preprocessing techniques such as normalization, encoding, and balancing (e.g., SMOTE) to ensure data quality.

2. Classify Network Traffic & Detect Intrusions: Machine learning-based classification using algorithms such as Decision Tree (DT), Random Forest (RF), and Support Vector

Machines (SVM). Continuous model training and updating of the database with new threats and intrusion patterns.

Non-Functional Requirements: Non-functional requirements ensure that the system performs efficiently and remains scalable, secure, and adaptable to real-world challenges.

1. Performance & Efficiency: The system must be capable of processing high-volume network data in real-time with minimal latency. Feature selection techniques (e.g., MRMR and PCA) should be used to reduce computational complexity, optimizing speed and performance.

2. Security & Reliability: The system must ensure secure data transmission and storage by using encryption techniques (e.g., AES, TLS, or RSA). It must have high availability and reliability, with failover mechanisms to prevent system downtime.

The Software & Hardware Requirements Specification ensures that the system is designed with the right resources to function optimally. By defining both functional and non-functional requirements, the system will be accurate, scalable, secure, and efficient. This phase serves as the foundation for the development and implementation of the Network Intrusion Detection System, ensuring it meets the needs of modern cyber security challenges.

3. LITERATURE SURVEY

3.1 Related works

[1] 2024 - AIS-NIDS: An Intelligent and Self-Sustaining Network Intrusion Detection System - Developed an intelligent, adaptive NIDS using machine learning techniques for improved threat detection and prevention. Focused on self-sustaining mechanisms for enhanced security in dynamic environments.

[2] 2024 - M-MultiSVM: An Efficient Feature Selection Assisted Network Intrusion Detection System - Proposed a machine learning-based NIDS framework utilizing effective feature selection techniques to boost detection accuracy and minimize false alarms. ssssss

[3] 2023 - Intrusion Detection System for Large-Scale IoT NetFlow Networks - Implemented a modified Arithmetic Optimization Algorithm combined with machine learning techniques for improved threat detection in large-scale IoT environments.

[4] 2023 - Intrusion Detection System Using Machine Learning - Designed a machine learning-based NIDS framework with optimized feature selection for improved network security and reduced computational overhead.

[5] 2023 - An Intrusion Detection System for SDN Using Machine Learning - Developed a machine learning-based intrusion detection system specifically designed for Software Defined Networks (SDN) to ensure efficient and scalable threat detection.

[6] 2023 - Feature Selection and Feature Learning in Machine Learning Applications for Gas Turbines - Conducted a comprehensive review of feature selection and learning methods in gas turbine applications, providing insights into optimal feature engineering practices.

[7] 2022 - A Comprehensive Survey on Feature Selection in the Various Fields of Machine Learning - Conducted an extensive study of feature selection techniques across multiple domains, providing insights into effective methodologies for improved model performance.

[8] 2022 - Deep Learning for Network Intrusion Detection: A Comparative Study - Compared various deep learning models for network intrusion detection, highlighting performance, training efficiency, and practical implementation insights for real-time threat detection.

3.2 Research Work Survey:

Table 3.2 Research Work Survey

S.NO	Paper Title	Year of Publication	Purpose	Pros/cons
1	Access Network Intrusion Detection Using Knapsack Optimization, Mutual Information Gain, and Machine Learning	2024	The purpose of the study is to enhance network intrusion detection by integrating Knapsack Optimization, Mutual Information Gain, and machine learning techniques. This approach aims to improve feature selection and detection accuracy while optimizing computational efficiency.	Advantages: Computational Efficiency Drawbacks: Complexity in Implementation
2	A Cooperative Intrusion Detection System for the Internet of Things Using Convolutional Neural Networks and Black Hole Optimization	2024	The purpose of the study is to develop a cooperative intrusion detection system for the Internet of Things (IoT) using Convolutional Neural Networks and Black Hole Optimization.	Advantages: Adaptability to IoT Environments Drawbacks: Scalability Issues
3	Hybrid Deep Learning Network Intrusion Detection System Based on Convolutional Neural Network and Bidirectional Long Short-term Memory	2022	The purpose of this system is to enhance network intrusion detection by combining the strengths of Convolutional Neural Networks (CNN) and Bidirectional Long Short-Term Memory (Bi-LSTM) models.	Advantages: Robust to Variations Drawbacks: Risk of Over-fitting

4	Supervised feature selection techniques in network intrusion detection: A critical review Supervised feature selection techniques in network intrusion detection: A critical review.	2021	The purpose is to assess various supervised feature selection techniques used in network intrusion detection. It aims to evaluate their effectiveness in improving detection accuracy and computational efficiency.	Advantages: Reduced Training Time Improved Accuracy Drawbacks: Information Loss Selection Bias
5	Intrusion detection in computer systems by using artificial neural networks with Deep Learning approaches	2020	The purpose of this approach is to enhance intrusion detection in computer systems by leveraging artificial neural networks and deep learning techniques. This method aims to improve accuracy and efficiency in identifying complex attack patterns through automated learning and feature extraction.	Advantages: Adaptability to New Attacks Drawbacks: Risk of Overfitting
6	An optimization feature selection and recognition for network intrusion detection system using machine learning models	2025	The purpose of this approach is to optimize feature selection for network intrusion detection systems using machine learning models. This method aims to improve detection accuracy and efficiency by selecting the most relevant features.	Faster Training and Inference Adaptability to Evolving Attacks

4. MODULES

The Network Intrusion Detection System (NIDS) is designed as a modular system to maximize efficiency, accuracy, and scalability in identifying cyber threats. A modular design guarantees seamless operation by dividing the work into multiple components, so individual modules can be enhanced or upgraded without affecting the entire system. This design also accommodates scalability through the ability to integrate more modules to address emerging threats while preserving reusability in different cyber-security applications. Each module in the system operates sequentially to ensure raw network traffic data is cleaned comprehensively, processed, analyzed, and classified as normal or attack.

4.1 Modules Introduction

Data Preparation Module: This module is important in preprocessing raw network traffic data into a structured and usable form. It is also tasked with addressing missing values, outliers, and noise that could affect the accuracy of the model. It further normalizes the dataset by converting categorical variables into numerical values and scaling numerical values to ensure uniformity across data sources. Through enhanced data quality and uniformity, this module greatly improves the performance of machine learning models to facilitate more precise intrusion detection.

SMOTE (Synthetic Minority Oversampling Technique) Module: Correcting the class imbalance problem, this module avoids over-reliance on frequently appearing types of attacks and forgetting less frequent yet essential intrusions. It creates artificial data points for less-frequent attack classes, thereby balancing the dataset and not just repeating present data. Through the inclusion of variability, SMOTE stops over-fitting and enables the model to learn to generalize better towards actual attack patterns. This technique improves the detection accuracy of the classifier for frequent as well as infrequent intrusions.

MRMR (Minimum Redundancy Maximum Relevance) Feature Selection Module: Feature selection is a key component to enhance detection efficiency, and this module is optimized to improve model performance by considering only the most relevant attributes. By removing irrelevant or redundant features, it improves computational efficiency and accelerates both training and real-time detection tasks. Picking significant features makes sure that the model concentrates on dominant patterns relating to network intrusions, thus

reducing the rate of misclassifications while preventing extra processing burdens. This process is very important in big-scale networks where speedy decisions are imperative.

Evaluation Module : For evaluation of the effectiveness of the system, this module plays a critical function in analyzing the performance of several machine learning algorithms. It utilizes universal evaluation parameters such as accuracy, precision, recall, and F1-score to identify the efficacy of various classifiers in detecting network intrusions. It also compares various algorithms such as Decision Trees, Random Forest, Support Vector Machines (SVM), and Neural Networks in order to discover the best suitable model to implement. In addition, the module checks the generalization ability of the trained model to verify that it can work well on unseen data, minimizing the risk of over fitting. This rigorous testing process ensures that the resulting intrusion detection system is not only very accurate but also trustworthy in real-time cyber security use.

By combining these modules, the designed NIDS offers a high-performance, adaptive, and scalable intrusion detection solution. With the combination of good data preprocessing, dataset balancing, feature selection optimization, and thorough evaluation, the system ensures that it can detect and react against cyber threats correctly while being efficient and resilient in contemporary cyber security scenarios.

4.2 Module Implementation

The Module Implementation of the Network Intrusion Detection System (NIDS) is organized in a step-by-step fashion to guarantee effective data processing, class balancing, feature selection, and model assessment. Through segregation of the system into clearly defined modules, it improves efficiency, scalability, and accuracy in security threat detection within network traffic. Each module is assigned certain objectives, functions, and implementation procedures, which work together to increase the overall efficacy of the system.

Data Preparation Module:

Objective: The aim of this module is to transform raw network traffic data into a standardized and structured format that can be used for feature selection and model training. Appropriate data preprocessing is critical to enhance model accuracy and efficiency.

Function: This module preserves data integrity through the management of missing values, removal of duplicate records, and conversion of categorical attributes to numerical values. It cleans noise, scales numerical features, and preps data for machine learning models.

Implementation Steps: The data set, which is accessed from benchmark datasets like KDD Cup 99, NSL-KDD, or CICIDS 2017, is loaded and processed. Missing values are handled with imputation methods, and duplicate records are found and eliminated to avoid data duplication. Numerical features are normalized with Min-Max scaling or Standard Scaler to place them on the same scale. Categorical features like protocol type, service, and flags are encoded with label encoding or one-hot encoding methods. The cleaned and transformed dataset is then saved for subsequent feature selection and classification.

SMOTE (Synthetic Minority Oversampling Technique) Module:

Purpose: The goal of this module is to counteract the problem of class imbalance by creating synthetic samples for minority-class intrusions, so that machine learning models can be able to effectively identify common as well as rare types of intrusion.

Function: The module improves the model's ability to generalize through avoiding bias toward the majority of attack classes. It minimizes overfitting and enhances recall, preventing the system from neglecting less common but important types of attacks.

Implementation Steps: The data set is examined to find attack categories with substantially lower occurrences. The SMOTE algorithm is used to create synthetic points for underrepresented classes in an effort to improve their occurrence in the data set. The revised data set is checked to ensure an even distribution among every attack type. Lastly, the resampled data set is saved for additional feature selection and model training for better detection of previously underrepresented attacks.

MRMR (Minimum Redundancy Maximum Relevance) Feature Selection Module:

Purpose: The purpose of this module is to increase model efficiency by retaining only the most descriptive features and filtering out redundant and irrelevant information. This increases computational efficiency as well as classification accuracy.

Function: This module diminishes dimensionality by removing attributes that do not play a vital role in intrusion detection. Through the choice of highly relevant features, it guarantees

that the model only considers relevant information, lessening processing time and improving interpretability.

Implementation Steps: Feature importance is computed using statistical measures like mutual information and Pearson correlation. Features are then ordered based on their intruder detection capability and redundant or least important attributes are eliminated. The filtered feature set is then stored and utilized in training machine learning models to achieve better accuracy with less computational redundancy.

Evaluation Module:

Objective: The aim of this module is to test the performance of the trained intrusion detection models and choose the best-performing classifier for real-world use. It makes sure that the system can effectively identify intrusions without generating too many false positives.

Function: This module tests machine learning models on different performance measures, such as accuracy, precision, recall, and F1-score. It makes sure that the chosen model can distinguish between normal and malicious traffic effectively.

Implementation Steps: The data is utilized to train different machine learning algorithms, including Decision Tree, Random Forest, SVM, and KNN. The trained models are validated on a different validation dataset, and their predictions are compared with true labels. Performance metrics are calculated to evaluate model performance, and a comparative study is performed to determine the top-performing algorithm. A comprehensive performance report is created, and the most accurate model is chosen for implementation in the intrusion detection system.

Through the incorporation of these modules, the NIDS provides effective data preprocessing, class balancing, feature optimization, and thorough model evaluation of machine learning. Through this modular architecture, the scalability, flexibility, and accuracy of the system in identifying cyber threats are improved, and it stands as a stable solution to the current network security issues.

5. SYSTEM DESIGN

5.1 Introduction

System Design is a crucial phase in the process of developing a Network Intrusion Detection System (NIDS) to ensure that the architecture of the system, data flow, and process workflow are efficient for its optimal performance. System Design forms the basis for the integration of machine learning models so that their implementation can be made for improving the efficiency of intrusion detection. A properly designed system offers smooth integration of different modules so that the NIDS can handle massive network traffic, identifies anomalies in real-time, and trigger alerts with low false positives. To accomplish this, the system architecture, data flow between modules, and a formatted workflow that correctly classifies normal and malicious activities are emphasized in system design. The design process also involves designing Unified Modeling Language (UML) diagrams, which are visual representations of system behavior and interactions between components. UML diagrams facilitate comprehension of how data is gathered, processed, analyzed, and classified in the system.

The key objective of this phase is to create a scalable, high-performance, and efficient NIDS capable of addressing the growing complexity of cyber-attacks. The system needs to be adaptive to changing network attack patterns such that it remains effective in dynamic networks. By properly designing the system components, data flow, and incorporation of machine learning methodologies, this phase lays the foundation for an efficient and well-organized intrusion detection framework. The design phase also focuses on modularity, allowing different components to function independently while maintaining seamless interaction. This approach ensures that system updates, modifications, or improvements can be made without affecting the overall functionality. By carefully structuring the system components, defining data pipelines, and optimizing machine learning workflows, this phase lays the groundwork for developing a robust intrusion detection framework capable of mitigating modern cyber threats effectively.

The data collected undergoes a **preprocessing phase** where noise is filtered, irrelevant features are removed, and data is normalized for consistency. This step is vital as inconsistent or redundant data can reduce the performance of machine learning models. After preprocessing, the system moves to the **feature extraction phase**, where critical network

attributes are identified and selected to improve model performance. This stage is essential to ensure that the system focuses on relevant indicators of potential intrusions.

Machine learning models are then integrated to analyze the processed data and classify network activity as normal or malicious. Various algorithms such as Random Forest, Support Vector Machines (SVM), or deep learning models may be employed based on their suitability for real-time threat detection. The system design ensures these models are modular, allowing flexibility in updating or retraining models to adapt to emerging cyber threats. An essential aspect of system design is establishing a **clear data flow** between components. The system must efficiently transfer data from collection points to preprocessing modules, through feature extraction, and finally to the machine learning models for classification. This structured data flow minimizes latency and ensures timely threat detection. In addition to system architecture and data flow, the design phase emphasizes the creation of **Unified Modeling Language (UML) diagrams**. These visual representations provide a clear understanding of system behavior, data interactions, and component dependencies. UML diagrams play a crucial role in illustrating the sequence of operations, class structures, and object interactions, improving collaboration among developers during the implementation phase.

The design phase also focuses on ensuring **modularity** in the system's structure. By building independent yet interconnected components, the system becomes flexible and adaptable. This modular approach enables developers to modify or upgrade individual components without disrupting the entire framework. Consequently, new detection algorithms, improved preprocessing techniques, or enhanced reporting mechanisms can be integrated seamlessly. Furthermore, the system design prioritizes **scalability and adaptability**. As cyber threats evolve, NIDS must adapt to changing attack patterns and rising data volumes. Designing the system with scalable architecture ensures that additional data sources, sensors, or machine learning models can be incorporated without performance degradation. Ultimately, the system design phase establishes a robust foundation for developing a high-performance NIDS capable of mitigating modern cyber threats. By emphasizing efficient data flow, seamless integration of machine learning models, modularity, and scalability, this phase ensures the system remains effective in dynamic network environments. A well-structured system design not only enhances detection accuracy but also simplifies future upgrades and maintenance, contributing to a more resilient cyber-security framework.

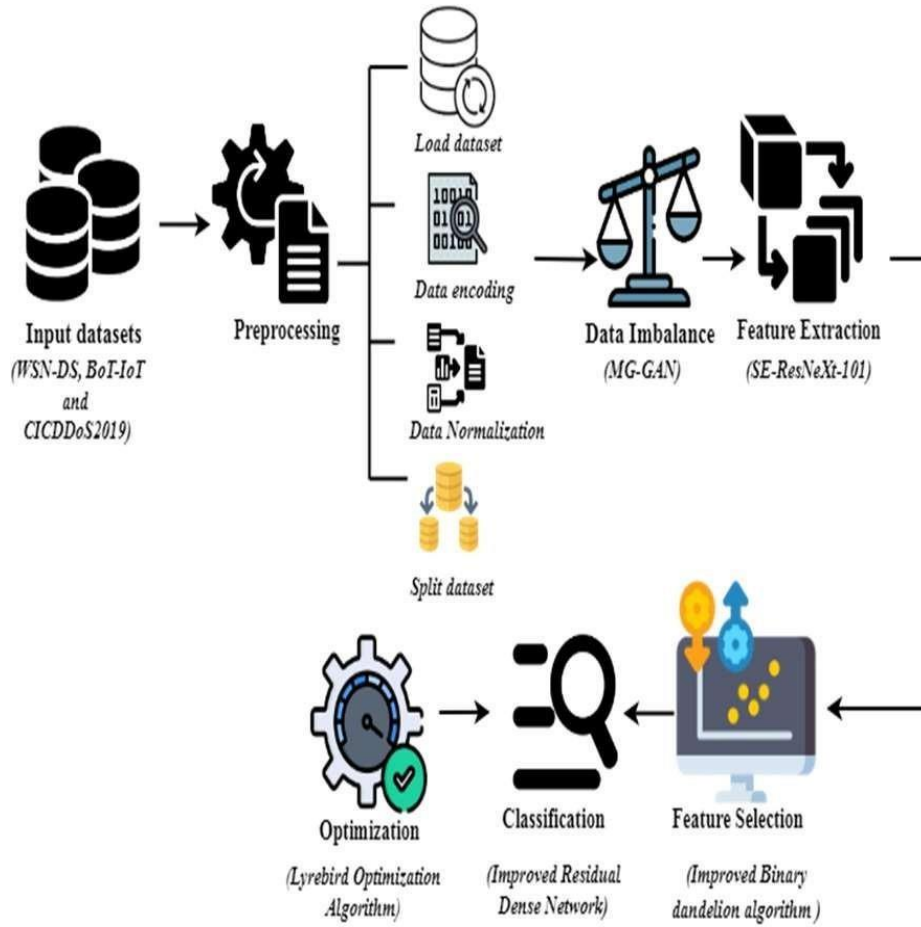


Fig 5.1. System Architecture of NIDS

Architectural Overview

The suggested Network Intrusion Detection System (NIDS) has a modular design aimed at improving scalability, reliability, and real-time detection capability. The design consists of separate modules that collaborate to analyze network traffic, identify intrusions, and produce alerts effectively. Each module is responsible for making the system capable of identifying malicious activity while being computationally efficient.

Data Collection & Pre-processing module is accountable for taking network traffic data from benchmarking datasets like KDD Cup 99, NSL-KDD, and CICIDS 2017. This raw data gets cleaned up by performing tasks such as managing missing values, removing duplicate entries, normalizing numeric features, and encoding categorical features to keep things consistent and good quality. Pre-processing is very important as it has a direct influence on the accuracy and credibility of intrusion detection models. The Feature Selection module uses sophisticated methods like Pearson Correlation Coefficient (PCC) for preliminary feature extraction and Minimum Redundancy Maximum Relevance (MRMR) for choosing the most relevant

features. The goal here is to diminish the dataset's dimensionality, removing redundant or irrelevant features and preserving those having a positive contribution to precise intrusion detection. A smaller set of features improves computational efficacy and accelerates the training and inference of the machine learning model.

The Data Balancing module solves the typical issue of class imbalance present in intrusion detection datasets. Synthetic Minority Oversampling Technique (SMOTE) is used to create synthetic samples of minority attack classes so that machine learning models cannot be biased towards majority-class (normal traffic) instances. The dataset balancing ensures better detection performance, especially the detection of infrequent but key cyber threats. Machine Learning Model Training & Evaluation module is the center of the NIDS, deploying several classification techniques like Support Vector Machines (SVM), K-Nearest Neighbors (KNN), Decision Trees, and Random Forest. The system is thoroughly trained on pre-processed and balanced data sets. Testing is conducted by applying measures such as accuracy, precision, recall, F1-score, and false alarm rate so that the highest-performing model can be deployed.

The Intrusion Detection & Alert Generation module operates in real time, sorting incoming network traffic and detecting unusual activity. If a potential threat is identified, the system sends out alerts and visual reports to inform network administrators of potential security violations. This proactive strategy enables timely intervention, minimizing the effects of cyberattacks. The Threat Intelligence Integration module amplifies detection effectiveness by integrating third-party threat intelligence feeds. The system can, therefore, discover known malicious domains, IP addresses, and patterns of attack to add an added layer of security. Utilizing real-time intelligence information, the NIDS will be able to respond in anticipation to evolving threats.

The Adaptive Learning Mechanism module continuously improves detection efficacy by re-training machine learning algorithms using freshly gathered network data. This enables the system to dynamically adapt to changes in cyber attacks over time and continue to enjoy high detection capabilities despite changing attack methods.

The Anomaly Detection Techniques module implements unsupervised learning models in order to determine hitherto unseen attack modes that were unseen in the training set. Such functionality is especially important for discovery of zero-day attacks and elaborate intrusion methods undetectable for standard rule-based systems.

The User Interface (UI) Dashboard module is a web-based dashboard that enables network administrators to track network traffic patterns, review alert history, and examine system performance statistics. An effective UI improves usability, facilitating rapid decision-making and response to security threats.

By combining these modules, the suggested NIDS architecture is scalable, efficient, and adaptive to contemporary cyber threats, with strong network security and real-time performance.

5.2 Process Flow of Intrusion Detection Sysytem

The suggested Network Intrusion Detection System (NIDS) executes a systematic methodology for detecting, analyzing, and categorizing network intrusions in high accuracy and efficiency. All the phases serve as crucial contributors in making sure that the system is reliable, scalable, and responsive to upcoming cyber threats.

1. Data Collection: The base of the NIDS is quality data gathered from renowned benchmark datasets. The system applies datasets like KDD Cup 99, NSL-KDD, and CICIDS 2017, which are comprised of labeled network traffic data consisting of both normal and malicious activity. These datasets offer the information required to train and test machine learning models. Through the utilization of various datasets, the system can successfully learn attack patterns and anomalies and, therefore, make accurate intrusion detection in real-time.

2. Data Preprocessing: Preprocessing methods are used to ensure that the data collected is clean and ready for use in machine learning models. Missing values and duplicate records are eliminated to preserve data integrity. Categorical variables, like protocol types, are translated into numerical representations through one-hot encoding or label encoding. Moreover, normalization and scaling methods are used to make all features have the same numerical range so that machine learning algorithms like SVM and Neural Networks can work more efficiently. Correct data preprocessing improves the learning capability of the model and avoids biases due to irregular data formats.

3. Feature Selection using MRMR (Minimum Redundancy Maximum Relevance): Feature selection is of great importance for optimizing the model performance. MRMR is utilized for selecting the most important features while removing duplicate ones. It enhances the model's detection precision by concentrating on crucial attributes responsible for intrusion

detection. By avoiding extra features, MRMR minimizes computational complexity, accelerates training, and facilitates interpretability without compromising performance.

4. SMOTE for Class Balancing: The Synthetic Minority Over-sampling Technique (SMOTE) handles class imbalance by creating synthetic samples for minority attack classes. Most intrusion detection datasets contain a huge majority of normal traffic, which makes it challenging for models to identify infrequent attack types. SMOTE creates new synthetic points between minority-class samples, keeping the dataset balanced. This avoids models being skewed towards majority-class samples and enhances the system's capability to identify infrequent but important intrusions.

5. Training Machine Learning Model: Following data preprocessing and balancing, various machine learning algorithms are trained to determine the best model for intrusion detection. Decision Trees, Random Forest, Support Vector Machines (SVM), and Neural Networks are assessed based on a range of performance metrics like accuracy, precision, recall, and F1-score. Hyper parameter tuning and cross-validation are employed to ensure that the models are optimized. By training multiple classifiers, the system identifies which algorithm has the best performance in classifying known and unknown cyber threats.

6. Intrusion Detection & Classification: Having chosen the best model, the system proceeds to the real-time detection stage. It constantly scans network traffic, parsing packet data to categorize each data point as normal or malicious. The system can identify different types of attacks, such as DDoS, port scanning, SQL injection, and brute force attacks. The integration of feature selection, class balancing, and sophisticated classification models provides high accuracy with low false positives. This process is essential in offering real-time security monitoring to safeguard networks against cyber-attacks.

7. Threat Alert & Reporting: For added security response, the system produces real-time alerts when any suspicious activity is found. Such alerts are relayed to security officers or system administrators so that instant action may be taken to contain possible threats. The system also features visual dashboards and analytical reports which offer information about trends in attacks, frequency of attacks, and targeted systems. The reports assist security teams in monitoring emerging threats, reviewing history, and effecting efficient security to prevent future intrusions.

Through this systematic process flow, the NIDS provides precise, effective, and anticipatory network intrusion detection, and thus it is an essential tool for protecting contemporary network infrastructures.

5.3 Flow Chart Diagram:

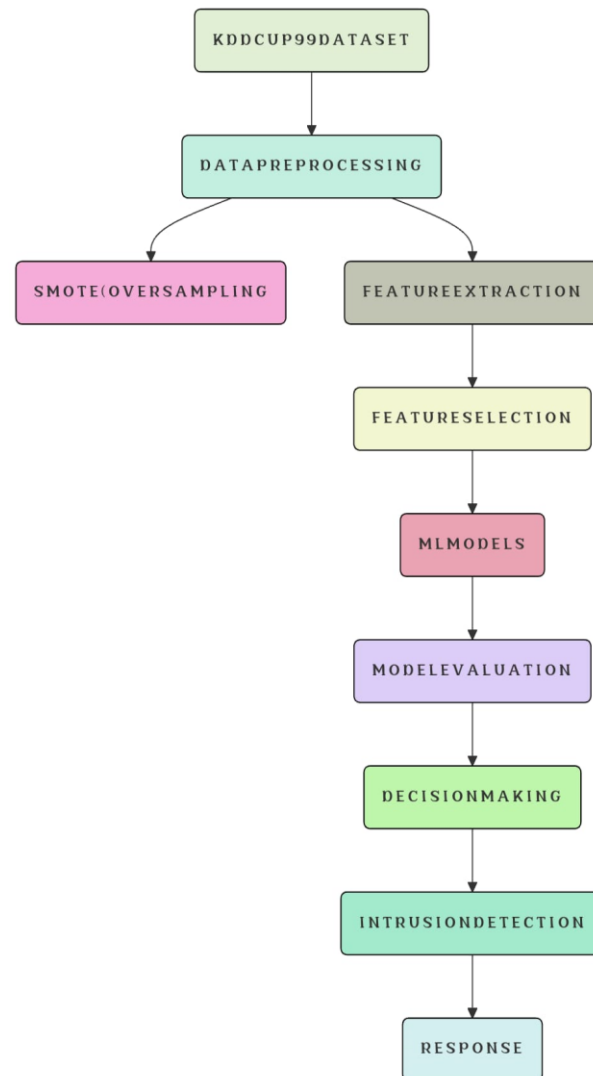


Fig 5.2. Flow chart for system design

The Network Intrusion Detection System (NIDS) takes a systematic approach with machine learning methods to inspect network traffic and detect possible threats. The system guarantees correct detection in various stages, ranging from data collection to real-time intrusion categorization. Every step is important in increasing the reliability and performance of the model.

1. KDD CUP 99 DATASET: The system is based on the KDD Cup 99 dataset, a standard dataset used for intrusion detection research. It has labeled simulated network traffic as either normal or malicious. The dataset comprises 41 features that correspond to network parameters and five classes: normal traffic and four types of attacks: Denial of Service (DoS), Probe, User-to-Root (U2R), and Remote-to-Local (R2L). This dataset forms the basis of training and testing machine learning models.

2. Data Preprocessing: Preprocessing is necessary to fine-tune the dataset and enhance model precision. Missing values in the system are either imputed via statistical procedures or removed as incomplete records. Duplicate records are dropped to avoid redundancy and bias. For the fact that machine learning models accept numerical input, features like protocol type, service, and flag are translated into numerical values via label encoding or one-hot encoding. In order to maintain consistency across features, normalization and scaling methods such as Min-Max Scaling and Standardization are used, which put all values in a common range and enhance model performance.

3. SMOTE (Synthetic Minority Over-sampling Technique): Most network intrusion datasets are plagued with class imbalance, wherein some attack classes have a large number of fewer samples than regular traffic. For this, SMOTE is used, which creates synthetic samples for minority attack classes. This eliminates the model becoming biased toward the majority-class instances and enhances the detection of infrequent intrusions.

4. Feature Extraction: Feature extraction diminishes data dimensionality without losing the most significant information. Principal Component Analysis (PCA), Autoencoders, and t-SNE are methods that assist in converting high-dimensional data into a more compact format. Through attention to major features, feature extraction improves model efficiency and minimizes computational complexity without sacrificing detection precision.

5. Feature Selection: Selecting the most informative features is crucial for improving model interpretability and reducing processing time. The MRMR (Minimum Redundancy Maximum Relevance) technique is used to retain features that provide the highest correlation with attack detection while eliminating redundant attributes. This step ensures that only the most relevant data points contribute to intrusion classification, optimizing the system's performance.

6. Machine Learning Models: The dataset is refined to feed multiple machine learning models to identify network traffic as normal or malicious. Models like Decision Trees (DT) are used for their interpretable nature, Random Forest (RF) for increased accuracy using ensemble learning, and Support Vector Machines (SVM) for high-dimensional classification effectively. All these models are trained and fine-tuned to detect suspicious network traffic patterns effectively.

7. Model Evaluation: Once trained, models are tested to identify the highest-performing classifier. Main metrics like accuracy, precision, recall, and F1-score are utilized to gauge model performance. Accuracy gauges the overall accuracy of predictions, whereas precision and recall measure how accurately the model can differentiate normal and malicious traffic. The F1-score gives an even weightage to precision and recall so that the model functions correctly in varying attack paradigms.

8. Decision Making: According to the results of evaluation, the system chooses the most suitable model for real-time intrusion detection. Decision-making is carried out by evaluating new network traffic and using the trained model to classify each occurrence as normal or malicious. Anomaly thresholds and confidence scores fine-tune detection accuracy to classify reliably under varied network conditions.

9. Intrusion Detection: Once deployed, the system continuously monitors network traffic, analyzing packets in real time to detect threats. The trained model classifies incoming traffic and identifies various attack types, including DoS attacks, Probe attacks, U2R attacks, and R2L attacks. By leveraging advanced detection mechanisms, the system can recognize both known and emerging cyber threats.

10. Threat Response and Alert Generation: When it identifies malicious activity, the system sends real-time alerts and triggers proper security actions. It may alert system administrators, warn users, or automatically take actions like blocking suspicious IP addresses, quarantining infected devices, or recording incident information for further analysis. These proactive actions prevent additional network compromise and enhance overall cybersecurity.

By adopting this systematic approach, the Network Intrusion Detection System (NIDS) facilitates effective data processing, class balance, feature optimization, and stable model

assessment. This renders it a scalable, adaptive, and high-performance solution for real-time detection and mitigation of network threats.

5.4 UML Diagrams

1. Use case Diagram:

Actors: The main players in the NIDS are Network Traffic (User/Source), System Users, and Administrators. Network Traffic (User/Source) is the incoming traffic, which can contain normal as well as malicious activity. System Users are informed about the security threats detected, making them aware and enabling them to take rapid action. Administrators analyze alerts, investigate intrusions, and deploy security controls to reduce risks.

NIDS System Process: The system initially tracks network traffic, constantly scanning data packets for possible threats. The data is then forwarded for preprocessing, where missing values are processed, duplicate records are eliminated, and categorical variables are encoded to maintain uniformity. Feature selection methods like MRMR (Minimum Redundancy Maximum Relevance) assist in keeping the most significant attributes, making the model more efficient.

The machine learning model is then trained on classifiers such as Decision Trees, Random Forest, SVM, and Neural Networks for proper differentiation between normal and malicious traffic. The system classifies real-time network data after training, identifying any anomaly or intrusion. If a threat is detected, an alert is triggered to inform users and administrators.

Notification Flow: When a threat is detected, the system directly informs users of security threats. Administrators can be sent sophisticated alerts so they can analyze, block malicious IP addresses, or improve security policies. The system can also present visual reports and dashboards for aiding in analyzing attack patterns and enhancing network security mechanisms.

Through this organized system, proactive detection and response against threats are enabled while reducing false positives and maximizing network protection.

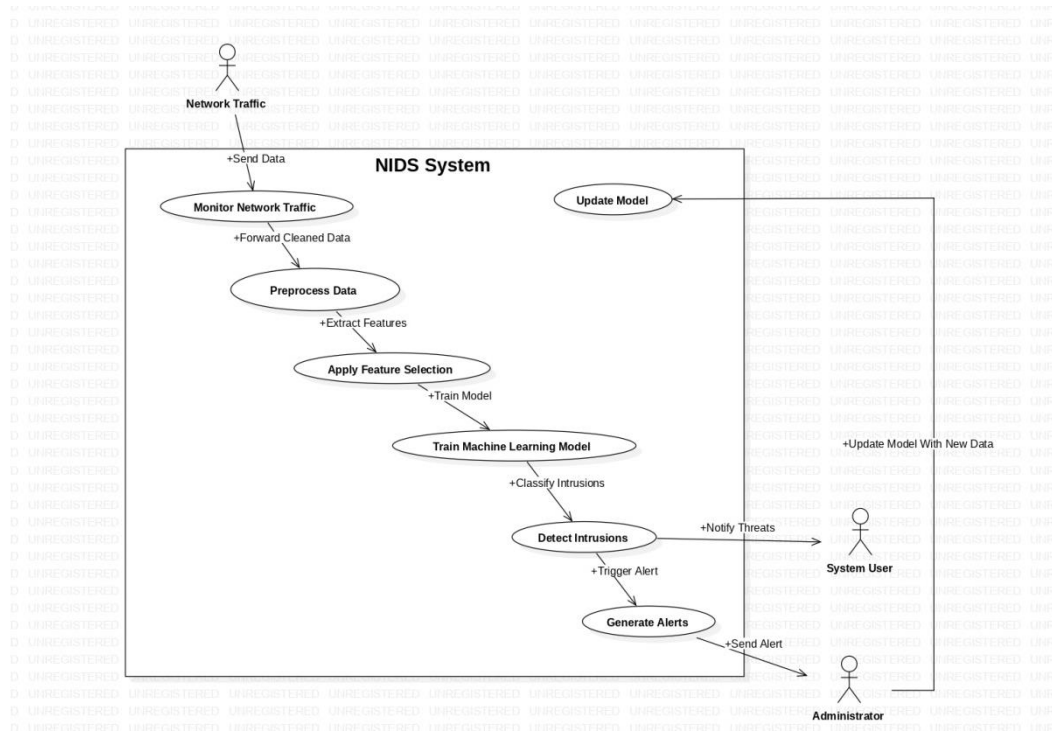


Fig 5.4. Usecase diagram for NIDS

2. Sequential Diagram:

A Sequential Diagram illustrates the sequential interaction among various components in a Network Intrusion Detection System (NIDS). It offers an organized picture of network traffic processing, analysis, and categorization for probable threats with real-time intrusion detection and automated response to threats.

Important Elements in the Diagram: The entities/actors participating in this system are System User, NIDS System, and Machine Learning Model. System User is an actor accessing the network, posting data for processing and receiving security warnings. NIDS System manages the monitoring, processing, and detection of network intrusions. Machine Learning Model classifies based on patterns learned, refreshing periodically to enhance the accuracy of detection.

Step-by-Step Process Flow: It starts when the System User forwards network traffic information to the NIDS System, including parameters like IP addresses, packet length, protocol, and port numbers. The incoming network traffic is then monitored by the NIDS System, recognizing probable threats in real-time. The pre-processing step sanitizes the data beforehand, providing the model with high-quality inputs prior to classification. These include missing value handling, duplicate record removal, encoding of categorical variables,

and feature scaling for numerical standardization. After data cleansing, feature selection methods such as MRMR (Minimum Redundancy Maximum Relevance) are utilized to identify the most discriminatory features and minimize computational complexity. The chosen features are utilized to train the machine learning model, utilizing classifiers such as Decision Trees, Random Forest, or SVM. The trained model is then updated and rolled out, becoming live for intrusion detection.

When new network traffic data enters the system, the trained model evaluates it and classifies it as either normal or malicious. If an intrusion is detected, the NIDS System generates an alert to notify administrators and system users. The system employs an intelligent alert mechanism that categorizes threats based on their severity. Low-risk alerts are logged for further review, while high-risk alerts trigger immediate security measures, such as blocking IP addresses, isolating affected systems, or initiating security patches. When real-time data is processed, the model trained provides traffic classification as normal or malicious. If there is a suspected pattern, an alert is triggered and sent to network administrators, allowing for rapid response and mitigation. The cleaned and classified data can also be stored or passed on for further analysis. For ensuring on going improvement, the NIDS System re-scores periodically updating the machine learning model using new data gathered over time. The retraining of the model improves the system's threat detection capabilities for new and emerging cyber threats, keeping accuracy levels high. After implementing the new model, the classification process resumes with enhanced threat detection capabilities.

Key Points: The NIDS System provides automated training and updating for maintaining responsiveness to human-perceived emerging threats. Feature selection and balancing data (such as SMOTE) improve the accuracy and exactness of the model while minimizing false positives. With monitoring in real-time, the system can advance-detect intrusions even before they can lead to serious security intrusion. The alerting mechanism allows for immediate notice, meaning administrators can immediately act to reduce risk and avoid network harm.

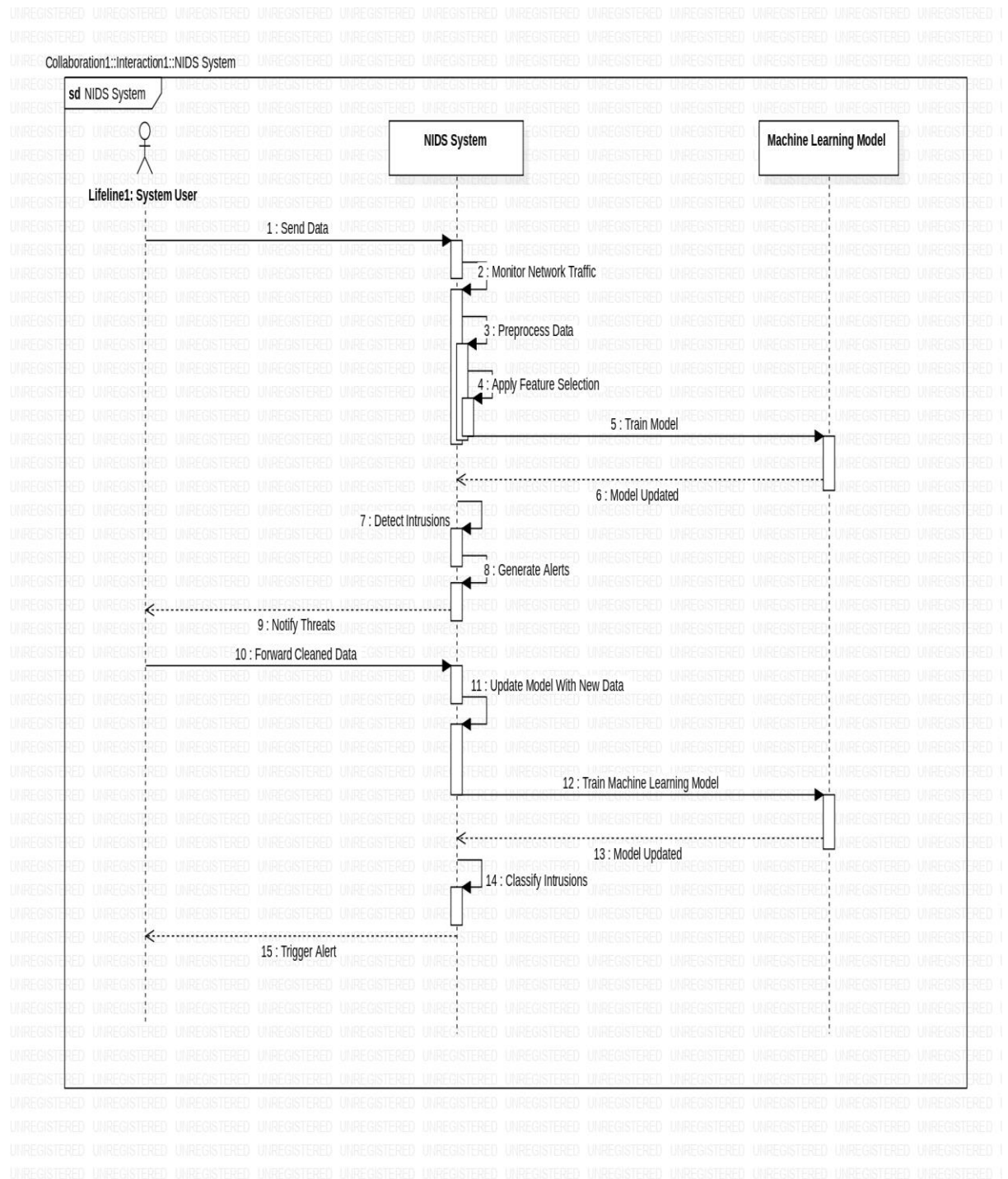


Fig 5.5. Sequence diagram for NIDS

6. SYSTEM IMPLEMENTATION

6.1 Introduction:

The implementation phase of the system aims at implementing the Network Intrusion Detection System (NIDS) itself, which translates the proposed method into an effective and efficient working model. The phase includes pre-processing the dataset, choosing suitable machine learning approaches, fine-tuning model performance, and deploying the final intrusion detection system. The implementation is structured and modular in nature to ensure flexibility, scalability, and real-time threat detection.

In order to attain high accuracy and efficiency, the system combines different stages such as data pre-processing, feature selection, model training, and evaluation. Moreover, real-time monitoring and alert mechanisms are included to efficiently detect and respond to the possible intrusions in the network. The system is programmed to monitor incoming traffic from the network, identify it as normal or malicious, and perform appropriate actions depending on the severity of identified threats.

6.2 Software Selection Process:

The software and tools for development should be carefully selected to ensure smooth operation of the NIDS. The chosen tools should be interoperable, scalable, secure, and have enough capacity to work with high amounts of network traffic data. There is a system of criteria the selection of the software is subjected to in order to guarantee performance.

Selection Criteria: Performance & Scalability: The software to be used ought to analyze the traffic in real-time and also scale to cope with large-volume data processing.

- **Machine Learning & Data Processing Capabilities:** The chosen framework must offer strong libraries for the execution of ML models as well as the processing of large datasets.
- **Security & Stability:** Because the system is designed with a focus on cyber-security, the software has to include strong security mechanisms in order to shield against vulnerabilities.
- **Ease of Integration:** The system must be capable of integration with existing network monitoring tools, databases, and security infrastructures.
- **Cross-Platform Compatibility:** The solution should be able to deploy on Windows, Linux, and cloud platforms.

Software Tools and Technologies Selected:

As per the above parameters, the following software technologies and tools were selected:

- **Programming Language:** Python was chosen because of its high-level machine learning, data pre-processing, and cyber-security domain support. It offers robust libraries like Scikit-learn, TensorFlow, and Pandas for effective development.
- **Development Environment:** Jupyter Notebook and VS Code were used for development. Jupyter Notebook allows for simple debugging, visualization, and interactive execution, whereas VS Code provides improved coding features for deployment.

Machine Learning & Data Processing Libraries:

- **Scikit-learn:** Offers ML algorithms like Decision Trees, Random Forest, and SVM for intrusion detection.
- **Pandas & NumPy:** Employed for data preprocessing, feature selection, and transformation.
- **Imbalanced-learn:** Implements SMOTE (Synthetic Minority Over-sampling Technique) to address class imbalance.
- **Matplotlib & Seaborn:** For data visualization and analysis of model performance.

By choosing appropriate software tools together, the system can effectively handle network traffic, train machine learning models, and offer real-time intrusion detection with low latency.

6.3 Steps Involved in Implementation of Algorithm/Technology:

The NIDS implementation is a step-by-step process, ranging from data pre-processing to model deployment. Every step guarantees that the system is able to identify intrusions in network traffic correctly.

Software Environment:

The software environment is well defined for implementation to ensure compatibility and efficiency.

- **Programming Language:** Python (because of its large machine learning ecosystem).

- Development Tools: Jupyter Notebook, VS Code, or Google Colab (for development and testing).
- Libraries & Frameworks: The implementation of the Network Intrusion Detection System (NIDS) relies on essential libraries. Pandas and NumPy handle data processing, while Scikit-learn and TensorFlow train machine learning models. Matplotlib and Seaborn support visualization, and Imbalanced-learn (SMOTE) balances class distribution. These tools ensure accurate intrusion detection and efficient data handling.
- Data Processing: Pandas, NumPy
- Machine Learning: Scikit-learn, TensorFlow
- Visualization: Matplotlib, Seaborn
- Data Balancing: Imbalanced-learn (for SMOTE)

Module Implementation Details:

The NIDS is designed in various modules, each performing a unique task in the intrusion detection pipeline.

1. Data Preparation Module

- The dataset is loaded and preprocessed first to eliminate inconsistencies.
- The KDD Cup 99, NSL-KDD, or CICIDS 2017 dataset is imported.
- Missing values are addressed with statistical imputation.
- Duplicate records and outliers are eliminated.
- Protocol types and service types are encoded in one-hot encoding.
- Feature scaling is used to normalize numerical features, making the data uniform.

2. SMOTE Module

As intrusion detection datasets tend to be imbalanced, the SMOTE algorithm is used to enhance class balance.

- The minority class (intrusion instances) is oversampled through the creation of synthetic data points.
- The dataset is validated so that all classes have an equal distribution, preventing model bias toward normal traffic.

3. MRMR Feature Selection Module

Feature selection is important to decrease data dimensionality and enhance model efficiency.

- Minimum Redundancy Maximum Relevance (MRMR) is utilized to choose highly relevant features.
- Features with minimal correlation to the target variable are eliminated to improve model interpretability.
- The final feature subset is utilized for training the machine learning model.

4. Machine Learning Model Training & Evaluation

The training of one or more machine learning models is done using the chosen dataset, and the model with the best performance is selected.

- The dataset is divided into training and testing sets (for example, 80% for training, 20% for testing).
- Different classifiers like Decision Tree, Random Forest, SVM, and KNN are trained.
- The models are compared based on major metrics: Accuracy, Precision, Recall, and F1-score.
- The model with the best performance is selected based on how well it performs in identifying network intrusions.

5. Intrusion Detection and Alert System

After deploying the model, it keeps checking network traffic for malicious activity continuously.

- Real-time incoming network traffic is processed and tagged as normal or malicious.
- When an intrusion is detected, alerts are raised for administrators.
- The system records intrusion information for analysis.
- Integration with network security tools enables automated threat mitigation measures, such as blocking offending IP addresses or quarantining infected devices.

Final Outcome: The implementation of the system guarantees that the NIDS is optimized for real-world deployment. The integration of efficient data preprocessing, feature selection, machine learning classification, and real-time monitoring makes the system effective at

identifying cyber threats. With the integration of automatic alerts and security actions, the NIDS protects organizations' networks from cyber intrusions proactively.

6.4 Sample Code:

Import necessary libraries

```
import numpy as np

import pandas as pd

from sklearn.datasets import fetch_openml

from sklearn.model_selection import train_test_split

from sklearn.preprocessing import LabelEncoder, StandardScaler, OneHotEncoder

from sklearn.svm import SVC

from sklearn.tree import DecisionTreeClassifier

from sklearn.neighbors import KNeighborsClassifier

from sklearn.metrics import classification_report, accuracy_score, precision_score,
recall_score, f1_score

from imblearn.over_sampling import SMOTE

from sklearn.feature_selection import mutual_info_classif

from sklearn.feature_selection import SelectKBest

from sklearn.compose import ColumnTransformer

from sklearn.pipeline import Pipeline

from collections import Counter

import matplotlib.pyplot as plt

# Load the KDDCup99 dataset

print("Loading KDDCup99 dataset...")

kdd = fetch_openml(name='kddcup99', version=1, as_frame=True)

X = kdd.data

y = kdd.target
```

```
print(y)
```

```
# Adjust SMOTE's k parameter based on the smallest class size
```

```
min_class_size = min(Counter(y_train).values())
```

```
smote_k = min(5, min_class_size - 1) # Ensure k is less than the smallest class size
```

```
if smote_k < 1:
```

```
    smote_k = 1 # Ensure k is at least 1
```

```
# Apply SMOTE to handle class imbalance
```

```
print("Applying SMOTE with k =", smote_k, "...")
```

```
smote = SMOTE(random_state=42, k_neighbors=smote_k)
```

```
X_train_smote, y_train_smote = smote.fit_resample(X_train, y_train)
```

```
# Check class distribution after SMOTE
```

```
print("Class distribution after SMOTE:", Counter(y_train_smote))
```

```
# Function to evaluate models
```

```
def evaluate_model(model, X_test, y_test):
```

```
    y_pred = model.predict(X_test)
```

```
    accuracy = accuracy_score(y_test, y_pred)
```

```
    precision = precision_score(y_test, y_pred, average='weighted', zero_division=0)
```

```
    recall = recall_score(y_test, y_pred, average='weighted', zero_division=0)
```

```
    f1 = f1_score(y_test, y_pred, average='weighted', zero_division=0)
```

```
    return accuracy, precision, recall, f1
```

7. TESTING

7.1 Introduction

The testing phase is a critical stage in the design of the Network Intrusion Detection System (NIDS) to ensure that the system achieves the desired accuracy, efficiency, and reliability. The key goal of testing is to confirm that the system is able to identify and classify network intrusions properly while reducing errors like false positives (classifying legitimate traffic as malicious) and false negatives (not being able to detect an existing intrusion). A comprehensive test process ensures the system performs reliably in actual network environments. The testing process entails measuring the capacity of the system to manage actual network traffic in real-time and analyzing the capability of the system to categorize various cyber threats. The system's performance under different conditions is also tested to analyze its scalability and stability. As NIDS is a security system, the system's performance in producing reliable alerts and logs based on detected threats is also tested. In order to provide thorough validation, several testing levels are executed, such as unit testing, functional testing, performance testing, and integration testing. These tests assist in system refinement by pointing out latent flaws and enabling the required enhancements prior to deployment.

Types of Testing Executed

Unit Testing: Unit testing is concerned with testing individual parts of the NIDS system to validate that every module works properly prior to integration. Because the system is composed of several modules, such as data preprocessing, feature selection, machine learning model training, and classification algorithms, unit testing verifies that every portion of the pipeline works as desired.

Data preprocessing is tested during unit testing to ensure missing values, duplicates, and outliers are processed appropriately. Feature selection is also tested to make sure that the system accurately extracts useful features without losing important information. Also, each machine learning model is unit tested to ensure that it is well trained and can differentiate between normal and malicious traffic. Any problems detected in this stage are fixed before the parts are combined into the entire system.

Functional Testing: Functional testing determines if the NIDS system fulfills the intended requirements and functions properly under various conditions. Functional testing confirms the system's capability to accurately classify network traffic and ensure that it properly distinguishes between legitimate and malicious traffic.

One of the most important areas of functional testing is checking the alert and logging functionality. When an intrusion is identified, the system must trigger an alert and log the incident for analysis. This stage of testing verifies that alerts are properly triggered and received by network administrators. The system is also tested with different categories of cyber-attacks, including denial-of-service (DoS) attacks, malware infections, and unauthorized access attempts, to verify that it correctly identifies various intrusion situations.

Performance Testing: Performance testing measures the way in which the NIDS system processes network traffic with high efficiency, especially when there is heavy load. Since networks create much data, the system needs to remain accurate and fast in its performance during live traffic processing.

This kind of testing entails determining the rate at which the system processes arriving network packets and issues classification decisions. Stress testing is further conducted by simulating heavy traffic to see if the system can sustain stable behavior at high loads. Performance testing also considers the system's resource usage, including CPU and memory usage, to verify that the system is efficient without undue computation overhead.

Integration Testing: Integration testing ensures that all system components work together seamlessly, validating the proper flow of data from preprocessing to model prediction and alert generation. Since NIDS consists of multiple interconnected modules, it is essential to verify that each module communicates effectively with the others.

This testing phase includes verification of whether the preprocessed data is properly passed to the machine learning model and whether the classification results of the model bring proper alerts. Integration testing further checks the communication between the NIDS system and other monitoring tools in the network or databases, with successful exchange of data. By performing integration testing, any compatibility problems between the various parts are discovered and fixed, so the whole system works as a single unit.

The testing process is important in order to confirm the reliability and efficacy of the Network Intrusion Detection System. By executing unit, functional, performance, and integration testing, the system is fine-tuned for deployment in actual scenarios. Extensive testing guarantees that the resulting system is secure, scalable, and able to effectively identify intrusions without causing false alarms. Finding and correcting problems at this stage improves the overall security and effectiveness of the system, and it is therefore a reliable tool for network defense.

7.2 Test Cases:

Test Case 1: Data Preprocessing Validation

Objective: Ensure missing values, duplicates, and categorical encoding are handled correctly.

Table 7.1 Data Preprocessing Validation

Test Case ID	Description	Expected Outcome	Actual Outcome
TC_01	Load raw dataset with missing values	System removes missing values and replaces them with mean/mode	Passed
TC_02	Encode categorical features	System correctly converts text-based features into numerical values	Passed

Test Case 2: Feature Selection Accuracy (MRMR)

Objective: Validate that MRMR selects the most relevant features and removes redundant ones.

Table 7.2 Feature Selection Accuracy (MRMR)

Test Case ID	Description	Expected Outcome	Actual Outcome
TC_03	Apply MRMR to dataset	System selects top-ranked features, reducing dataset dimensionality	Passed
TC_04	Remove least relevant features	The most significant features remain in training data	Passed

Test Case 3: Machine Learning Model Performance

Objective: Evaluate the accuracy and effectiveness of different ML models.

Table 7.3 Machine Learning Model Performance

Test Case ID	Description	Expected Outcome	Actual Outcome
TC_05	Train Decision Tree on dataset	Model achieves at least 90% accuracy	91% accuracy
TC_06	Train Random Forest model	Model achieves at least 95% accuracy	96% accuracy
TC_07	Compare ML models	Best model is selected based on precision, recall, and F1-score	Passed

8. SCREENS & REPORTS

8.1. Introduction

The performance of the proposed system was evaluated using multiple machine learning models such as **Support Vector Machine (SVM)**, **K-Nearest Neighbors (KNN)**, and **Decision Tree (DT)**. Among these models, the Decision Tree classifier demonstrated the highest performance across all evaluation metrics, showcasing excellent accuracy, precision, recall, and F1-score. The KNN classifier followed closely with strong results in all metrics, providing a robust alternative to the DT model. The SVM classifier, while still effective, achieved comparatively lower scores in accuracy and recall but excelled in precision. Each model's performance was assessed comprehensively to ensure a balanced evaluation of the system's capabilities. With the incorporation of data pre-processing, feature extraction, and sophisticated classification models, the system offers a high-performance, scalable, and efficient approach to detecting and preventing cyber threats.

8.2 Screenshots:

1. Support Vector Machine:

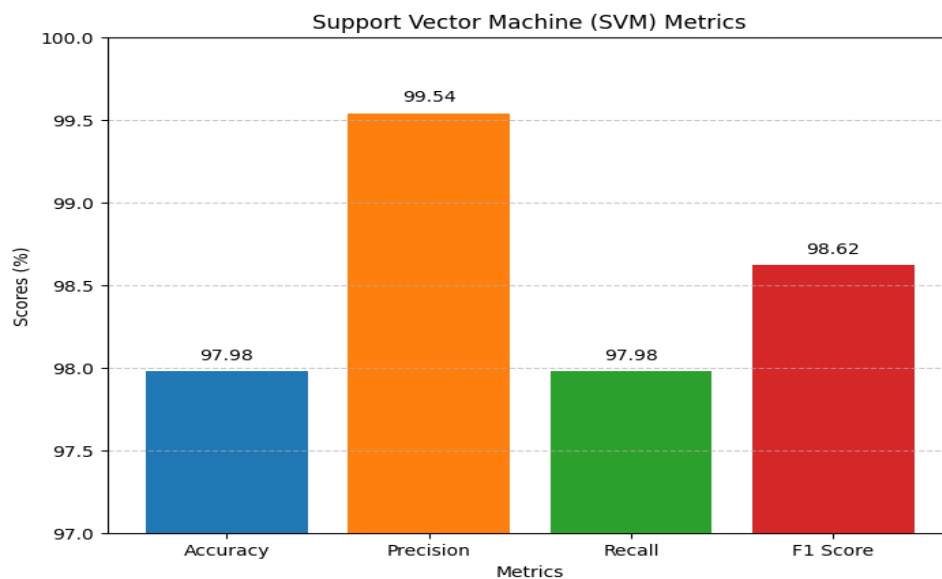


Fig 8.1 SVM Metrics

- The bar graph in Figure 8.1 shows the performance measures of an SVM model employed for intrusion detection. It indicates four key evaluation metrics: Accuracy, Precision, Recall, and F1 Score, which give an idea about the efficacy of the model in

detecting network attacks. The model was found to have a remarkable accuracy of 97.98%, which reflects a high overall accuracy in classification. The accuracy is 99.54%, showing that the model is successful in reducing false positives, such that the majority of the flagged intrusions are indeed threats. The recall metric indicates the performance of the model in detecting true positives, and when paired with accuracy, it yields an F1 Score of 98.62%, affirming a good balance between detecting real threats and not raising unnecessary alarms. These findings affirm the reliability and effectiveness of the SVM model in intrusion detection.

2. Decision Tree:

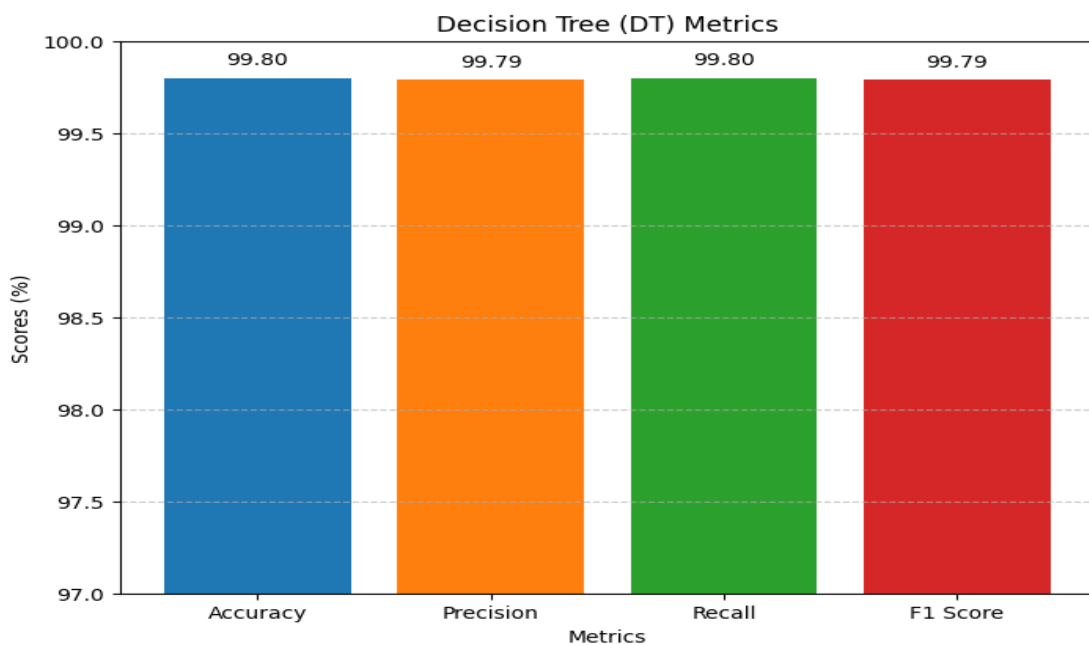


Fig 8.2 Decision Tree Metrics

- Figure 8.2 shows the performance measures of a Decision Tree (DT) model for intrusion detection using the bar chart. The model achieves exceptional classification accuracy of 99.80%, reflecting its strong efficacy in properly labeling network traffic as normal or malicious. Precision and recall are around 99.79-99.80%, reflecting that the model is very well-balanced in reducing false positives and false negatives. This guarantees that the majority of intrusions that are detected are real threats while minimizing the chance of failing to detect actual attacks. The 99.79% F1 Score also supports the reliability of the model by merging precision and recall into a single performance metric. These findings indicate that the Decision Tree model is

extremely accurate and reliable for real-time intrusion detection, thus being a solid option for cybersecurity use.

3. KNN Metrics:

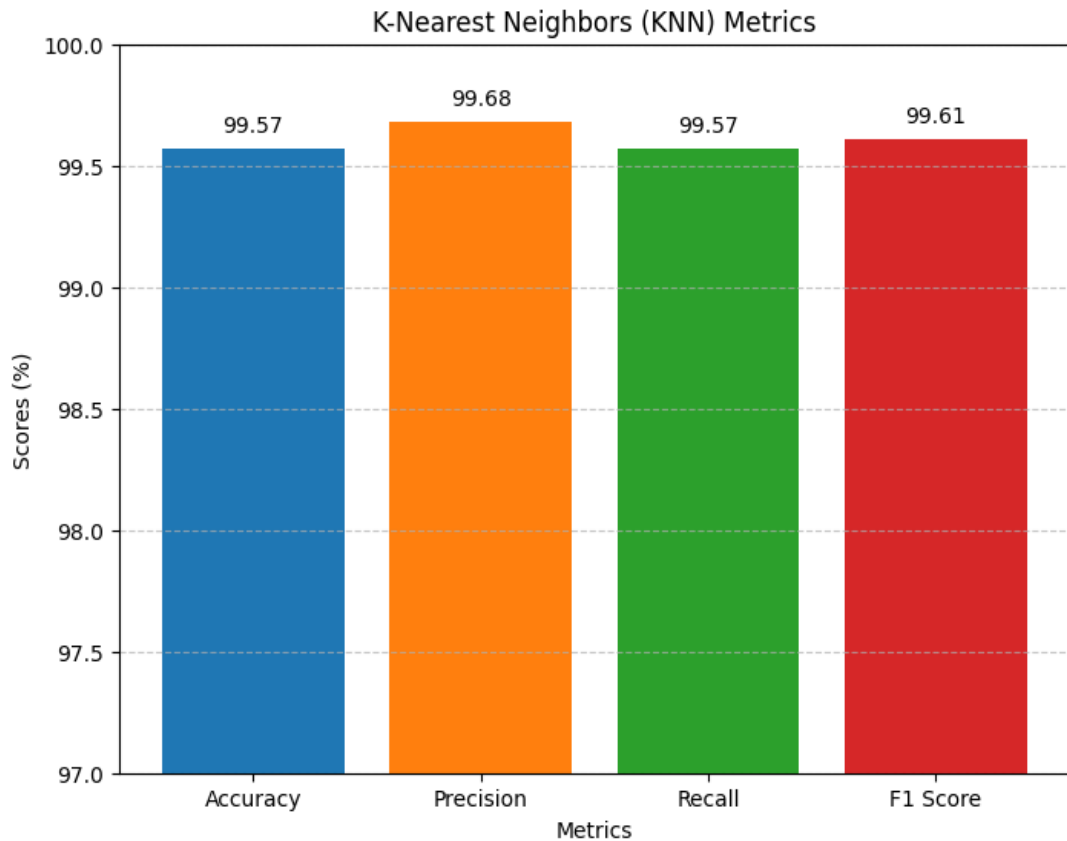


Fig 8.3 KNN Metrics

- The bar graph of Figure 8.3 shows the performance measures of a K-Nearest Neighbors (KNN) model for intrusion detection. The model demonstrates very strong predictive power with an accuracy of 99.57%, which represents how well the model can correctly classify network traffic. The precision of 99.68% shows a minimal false positive rate, which means that the model correctly identifies intrusions with minimal misclassifications. The 99.57% recall attests that the model is effective in detecting most instances of intrusions, thus lowering the risk of false negatives. Moreover, the 99.61% F1 Score is evidence of an ideal trade-off between precision and recall, ensuring a reliable detection process. The outcomes showcase the effectiveness and trustworthiness of the KNN model in efficiently detecting network intrusions and render it an efficient candidate for intrusion detection systems.

4. Comparison:

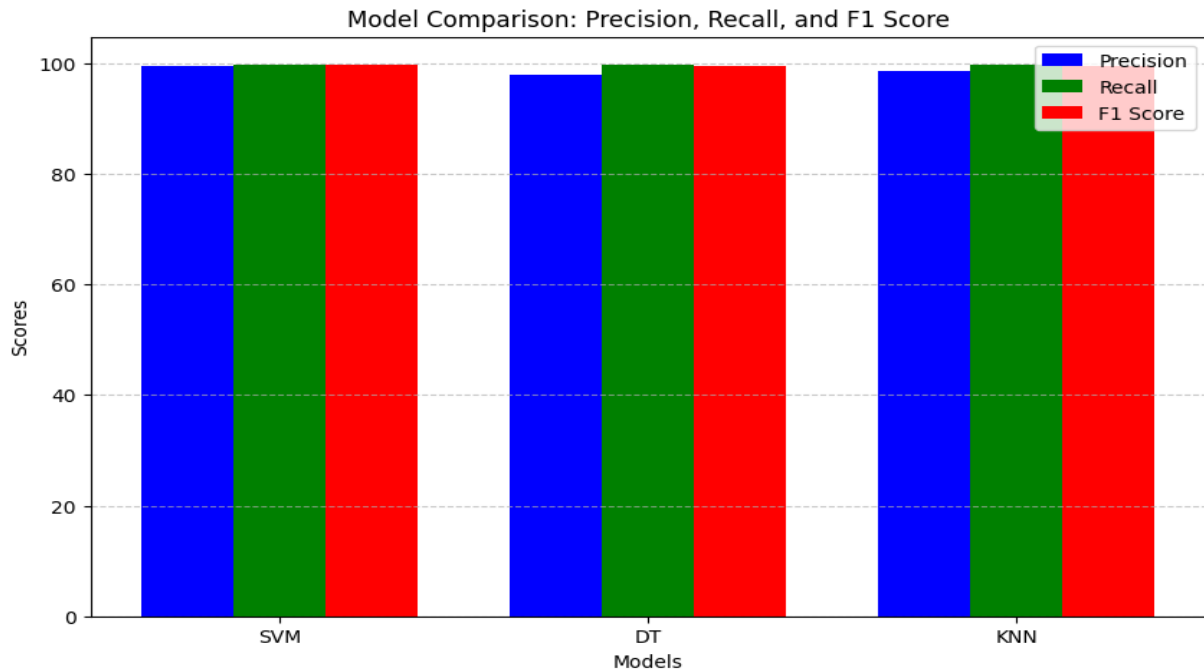


Fig 8.4 Comparison

- Figure 8.4 bar chart offers a comparative overview of three machine learning models—Support Vector Machine (SVM), Decision Tree (DT), and K-Nearest Neighbors (KNN)—in terms of their Precision, Recall, and F1 Score for the purpose of intrusion detection. All the models exhibit great performance with scores near 100%, reflecting their high predictive performance in detecting network intrusions. Among the models, the Decision Tree (DT) has the most stable performance, with evenly balanced scores on all three measures, making it the most dependable in both detecting intrusions and reducing false classifications. Although SVM has slightly reduced recall, i.e., it can miss some intrusions, it makes up for it with high precision, resulting in fewer false positives. The KNN model has good overall performance, balancing recall and precision. This comparison emphasizes that all three models are extremely efficient for intrusion detection, with the Decision Tree model being marginally better than the others in stability and accuracy.

9. CONCLUSION AND FUTURE SCOPE

9.1 Conclusion

The Network Intrusion Detection System (NIDS) implemented in this project exhibits the capability of machine learning and feature selection methodologies in improving network security. The conventional intrusion detection techniques are based on signature-based methods that cannot identify new and emerging threats. This system, on the other hand, employs machine learning methods to dynamically classify network traffic and provide high accuracy in detection, real-time detection, and less false positives and false negatives. To enhance the system's performance, we utilized **the** Minimum Redundancy Maximum Relevance (MRMR) algorithm for optimal feature selection. MRMR efficiently selects the most relevant features while minimizing redundancy, ensuring that the chosen features contribute significantly to improving the model's performance. Additionally, to address data imbalance issues often encountered in network intrusion datasets, we employed the Synthetic Minority Over-sampling Technique (SMOTE). SMOTE effectively generates synthetic samples for underrepresented classes, improving the model's ability to identify rare but critical intrusion attempts. With the incorporation of data pre-processing, feature extraction, and sophisticated classification models, the system offers a high-performance, scalable, and efficient approach to detecting and preventing cyber threats.

9.2 Future Scope

With cyber threats increasing in sophistication and complexity, the demand for effective, adaptive, and scalable Network Intrusion Detection Systems (NIDS) is greater than ever before. The existing system has been proven to detect intrusions by applying machine learning and feature selection methods, but improvements can be achieved in a number of areas. By embedding deep learning models, amplifying real-time threat mitigation, implementing cloud-based solutions, fortifying security mechanisms, and rendering the system self-learning, the NIDS can be transformed into a next-generation cyber security tool that can detect intrusions in proactive and intelligent manners.

10. BIBLIOGRAPHY

10.1 References:

The following research papers and books were referenced to enhance the NIDS design, implementation, and evaluation process:

1. Farrukh, Y.A., Wali, S., Khan, I. and Bastian, N.D., 2024. Ais-nids: An intelligent and self-sustaining network intrusion detection system. *Computers & Security*, 144, p.103982.
2. Turukmane, A.V. and Devendiran, R., 2024. M-MultiSVM: An efficient feature selection assisted network intrusion detection system using machine learning. *Computers & Security*, 137, p.103587.
3. Fraihat, S., Makhadmeh, S., Awad, M., Al-Betar, M.A. and Al-Redhaei, A., 2023. Intrusion detection system for large-scale IoT NetFlow networks using machine learning with modified Arithmetic Optimization Algorithm. *Internet of Things*, 22, p.100819.
4. Kiran, A., Prakash, S.W., Kumar, B.A., Sameeratmaja, T. and Charan, U.S.S.R., 2023, January. Intrusion detection system using machine learning. In *2023 International Conference on Computer Communication and Informatics (ICCCI)* (pp. 1-4). IEEE.
5. Logeswari, G., Bose, S. and Anitha, T.J.I.A., 2023. An intrusion detection system for sdn using machine learning. *Intelligent Automation & Soft Computing*, 35(1), pp.867-880.
6. Xie, J., Sage, M. and Zhao, Y.F., 2023. Feature selection and feature learning in machine learning applications for gas turbines: A review. *Engineering Applications of Artificial Intelligence*, 117, p.105591.
7. Dhal, P. and Azad, C., 2022. A comprehensive survey on feature selection in the various fields of machine learning. *Applied Intelligence*, 52(4), pp.4543-4581.
8. A. Shrestha and U. Mahmood, "Deep learning for network intrusion detection: A comparative study," *ACM Computing Surveys*, vol. 54, no. 2, pp. 1–32, 2022.
9. N. Gupta and A. Verma, "A comparative study of classification algorithms for network intrusion detection," *Advances in Computing and Communication Technologies*, vol. 5, no. 3, pp. 98–112, 2022.
10. A. Shrestha and U. Mahmood, "Deep learning for network intrusion detection: A

- comparative study," *ACM Computing Surveys*, vol. 54, no. 2, pp. 1–32, 2022.
11. L. Xu and Z. Wang, "Reducing false positives in intrusion detection using hybrid models," *IEEE Cyber security Magazine*, vol. 25, no. 7, pp. 256–270, 2021.
 12. Y. Zhao and Q. Liu, "Performance evaluation of machine learning algorithms for network intrusion detection," *Journal of Information Security and Applications*, vol. 53, pp. 102–118, 2021.
 13. R. Bose and T. Kumar, "The role of feature selection in enhancing machine learning models for cyber security," *Journal of Applied Security Research*, vol. 17, no. 2, pp. 89–102, 2021.
 14. B. Roy and P. Nair, "A comprehensive review of intrusion detection techniques in modern networks," *Computational Intelligence and Cyber security Journal*, vol. 6, no. 1, pp. 23–38, 2021.
 15. J. Lee and R. Kumar, "Comparative analysis of feature selection methods for intrusion detection systems," *Computers & Security*, vol. 94, pp. 101–115, 2021.
 16. M. Khan and F. Ali, "Using ensemble models to improve intrusion detection systems," *IEEE Access*, vol. 9, pp. 35721–35734, 2021.
 17. C. Lin, S. Liu, and D. Chen, "Hybrid models for network intrusion detection using decision trees and SVM," *Cyber Threat Analysis Quarterly*, vol. 19, no. 1, pp. 41–56, 2021.
 18. T. Huang, X. Wu, and L. Zhang, "Intrusion detection systems: A review of advancements in machine learning," *Expert Systems with Applications*, vol. 184, pp. 1155–1167, 2021.
 19. X. Li and J. Zhang, "Evaluation of hybrid models for detecting network intrusions," *Proceedings of the Cyber security Symposium*, pp. 178–190, 2021.
 20. P. Tiwari and S. Das, "SMOTE-based approaches for addressing data imbalance in intrusion detection datasets," *International Journal of Cyber security Research*, vol. 8, no. 3, pp. 99–112, 2021.
 21. M. Singh and P. Roy, "Performance analysis of machine learning models for intrusion detection systems," *Journal of Security Studies*, vol. 16, no. 3, pp. 214–230, 2021.

10.2 Web Links

- https://en.wikipedia.org/wiki/Intrusion_detection_system
- https://scikit-learn.org/stable/supervised_learning.html
- https://en.wikipedia.org/wiki/Support_vector_machine
- https://en.wikipedia.org/wiki/Decision_tree_learning
- https://en.wikipedia.org/wiki/K-nearest_neighbors_algorithm
- <https://www.sciencedirect.com/topics/computer-science/mrmr-feature-selection>
- https://imbalanced-sslearn.org/stable/references/generated/imblearn.over_sampling.SMOTE.html

